# SRINIVAS UNIVERSITY
# INSTITUTE OF ENGINEERING & TECHNOLOGY
**MUKKA, MANGALURU- 574 146**

**MICROSERVICES ARCHITECTURE & IMPLEMENTATION**

**MINIPROJECT REPORT**

**ON**

# " CAMPUS LIFE OS "

***Submitted in the partial fulfillment of the requirements for the award of the degree of***

**BACHELOR OF TECHNOLOGY**
**IN**
**COMPUTER SCIENCE AND ENGINEERING**

**Submitted By,**


| | |
|---|---|
| **DARSHA B SHETTY** | **01SU23CS052** |
| **H MEGHA** | **01SU23CS071** |


**UNDER THE GUIDANCE OF**

**Mr. YOGESH**

**IBM TRAINER**

**2025-2026**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

# ABSTRACT

Campus Life OS is an advanced full-stack web-based application developed to streamline and enhance the overall productivity and daily activities of students within a campus environment. In modern academic settings, students often depend on multiple platforms and tools to manage tasks such as assignments, schedules, notifications, communication, and other utilities. This fragmented approach leads to inefficiency, time consumption, and lack of proper organization. The primary objective of this project is to eliminate these challenges by providing a centralized, integrated, and user-friendly system that consolidates all essential student services into a single platform.

The system is designed using a modern technology stack, where the frontend is developed using Next.js integrated with React and TypeScript to ensure a responsive, dynamic, and scalable user interface. The backend is implemented using Node.js and Express, following a microservices architecture. In this approach, different functionalities such as authentication service, task management service, notification service, and other campus-related modules are developed as independent and loosely coupled services. This architectural design enhances scalability, flexibility, and maintainability of the system.

An API Gateway is utilized to manage communication between the frontend and backend microservices, ensuring efficient request handling and secure data flow. Additionally, Docker is used for containerization, allowing each service to run in isolated environments, making deployment easier and ensuring consistency across different platforms. This also supports scalability and simplifies the process of managing and updating individual services without affecting the entire application.

Campus Life OS offers a unified interface where students can efficiently manage their academic responsibilities, track tasks, receive notifications, and access various campus utilities in one place. The system is designed with a focus on usability, performance, and real-world applicability. By integrating multiple services into a cohesive platform, it significantly reduces complexity and improves user experience.

Overall, this project demonstrates the practical implementation of modern software engineering concepts such as microservices architecture, containerization, modular design, and full-stack development. It serves as a strong portfolio project while also providing a scalable and efficient solution to real-world student productivity challenges.

# CONTENTS


| | | |
|---|---|---|
| **Chapter 1** | **INTRODUCTION** | **1** |
| 1.1 | Overview of the Project | 1 |
| 1.2 | Problem Definition | 1 |
| 1.3 | Objectives of the Project | 1 |
| 1.4 | Scope and Applications | 2 |
| **Chapter 2** | **LITERATURE REVIEW** | **3** |
| 2.1 | Existing Systems | 3 |
| 2.2 | Proposed System | 3 |
| **Chapter 3** | **SYSTEM ANALYSIS AND DESIGN** | **4** |
| 3.1 | System Architecture | 4 |
| 3.2 | Hardware Requirements | 5 |
| 3.3 | Software Requirements | 5 |
| 3.4 | Use Case | 6 |
| **Chapter 4** | **METHODOLOGY** | **6** |
| 4.1 | Working Principle | 7 |
| 4.2 | Process Flow | 7 |
| 4.3 | Module Description | 8 |
| 4.4 | Error Handling and Validation | 9 |

| | | |
|---|---|---|
| **Chapter 5** | **EXECUTION** | **11** |
| 5.1 | Execution Steps | 11 |
| 5.2 | Compilation & Execution Commands | 12 |

| | | |
|---|---|---|
| **Chapter 6** | **RESULTS** | **13** |
| **Chapter 7** | **CONCLUSION AND FUTURE SCOPE** | 14 |
| 7.1 | Conclusion | 14 |
| 7.2 | Future Enhancements | 14 |
| **Chapter 8** | **REFERENCES** | 15 |

# CHAPTER 1

# INTRODUCTION

## 1.1 Overview of the Project

Campus Life OS is a full-stack web application designed to unify various campus functionalities into one platform. It includes features such as user authentication, task management, notifications, and other utilities that support student activities.

The frontend is developed using Next.js with React and TypeScript to create an interactive and responsive user interface. The backend is built using Node.js and Express, structured as multiple independent microservices. Each service handles a specific functionality and communicates through an API Gateway.

Docker is used for containerization, enabling easy deployment and scalability. The system is designed to be modular, allowing future enhancements and updates without affecting the entire application.

## 1.2 Problem Definition

Students face difficulties in managing their academic and personal tasks due to the use of multiple disconnected systems. Some common problems include:

- Lack of a centralized platform for managing tasks and notifications
- Difficulty in tracking assignments and deadlines
- Missing important updates due to scattered communication channels
- Time wasted switching between multiple applications
- Poor organization and reduced productivity

These issues highlight the need for a unified system that can efficiently manage all student-related activities in one place.

## 1.3 Objectives of the Project

The main objectives of Campus Life OS are:

- To develop a centralized platform for managing student activities
- To improve productivity by organizing tasks and schedules efficiently
- To provide real-time notifications and updates
- To ensure scalability and flexibility using microservices architecture
- To create a user-friendly and responsive interface
- To simplify access to various campus utilities

- To implement modern technologies and real-world development practices

## 1.4 Scope and Applications

The scope of Campus Life OS extends to various areas within a campus environment and can be further expanded in the future.

**Scope**

- Integration of multiple student services into one platform
- Scalable architecture to support increasing users
- Easy addition of new features and services
- Deployment across different platforms using Docker

**Applications**

- Task and assignment management for students
- Notification and alert system for important updates
- Campus information and utility access
- Academic activity tracking
- Can be extended for use in schools, colleges, and universities

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 Existing System

In the current academic environment, students use multiple independent applications to manage different aspects of their campus life. For example, learning management systems (LMS) are used for assignments, messaging apps for communication, and separate tools for task tracking and scheduling.

Some common existing systems include:

- Learning Management Systems (LMS) for academic content and assignments
- Mobile applications for communication and notifications
- Task management apps for personal productivity

However, these systems have several limitations:

- Lack of integration between different platforms
- Scattered information across multiple applications
- Inefficient task and time management
- Poor user experience due to switching between apps
- Limited customization for student needs

These drawbacks create a need for a unified platform that can handle multiple functionalities in one place.

## 2.2 Proposed System

In the current academic environment, students use multiple independent applications to manage different aspects of their campus life. For example, learning management systems (LMS) are used for assignments, messaging apps for communication, and separate tools for task tracking and scheduling.

Some common existing systems include:

- Learning Management Systems (LMS) for academic content and assignments
- Mobile applications for communication and notifications
- Task management apps for personal productivity

However, these systems have several limitations:

- Lack of integration between different platforms
- Scattered information across multiple applications
- Inefficient task and time management
- Poor user experience due to switching between apps
- Limited customization for student needs

# CHAPTER 3

# SYSTEM ANALYSIS AND DESIGN

## 3.1 System Architecture

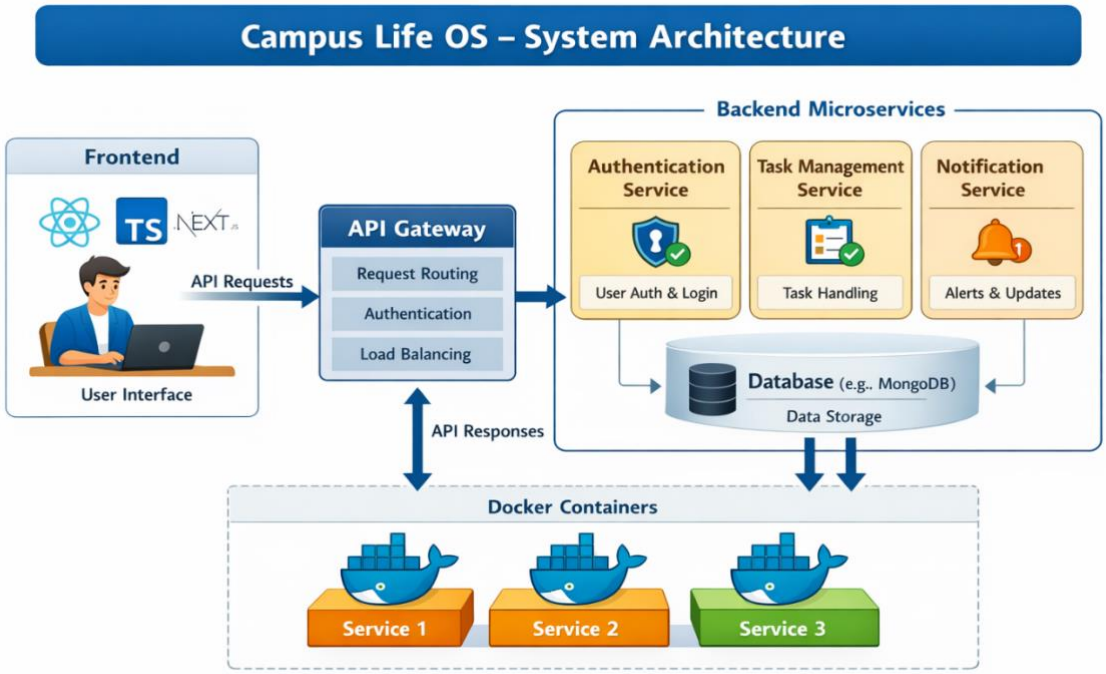
Campus Life OS follows a **microservices architecture**, where the application is divided into multiple independent services. Each service is responsible for a specific functionality such as authentication, task management, or notifications.

**Key Components:**

- **Frontend:** Developed using Next.js (React + TypeScript)
- **Backend Services:** Node.js and Express-based microservices
- **API Gateway:** Acts as a single entry point for all client requests
- **Database:** Stores user data, tasks, and other information
- **Docker:** Used for containerization and deployment

**Advantages of Architecture:**

- Scalability (services can be scaled independently)
- Flexibility (easy to modify or add features)
- Fault isolation (failure in one service does not affect others)




. **3.1.1 : System Architecture**

## 3.2 Hardware Requirements

The system requires minimal hardware resources for development and deployment.

**Development Requirements:**

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: At least 10 GB free space

**Deployment Requirements:**

- Cloud server or local machine
- Stable internet connection

## 3.3 Software Requirements

The software requirements for the project include:

**Development Tools:**

- Visual Studio Code (IDE)
- Node.js (Runtime environment)
- Docker (Containerization platform)

**Technologies Used:**

- Frontend: Next.js, React, TypeScript
- Backend: Node.js, Express
- API Communication: REST APIs
- Database: MongoDB (or similar NoSQL database)

**Operating System:**

- Windows / Linux / macOS

## 3.4 Use Case

The use case describes how the user interacts with the *Campus Life OS* system and the functionalities provided by the system. It represents the relationship between the user (student) and different system operations.

**Primary Actor**

- Student (User)

**Description**

The student interacts with the system to perform various activities such as registration, login, managing tasks, viewing notifications, and accessing campus services. The system processes user requests through backend services and provides appropriate responses.

**Main Use Cases**

**1. User Registration**

- The user creates a new account by providing required details such as name, email, and password.
- The system stores the user information securely in the database.

**2. User Login**

- The user logs into the system using valid credentials.
- Authentication service verifies the details and grants access.

**3. Manage Tasks**

- The user can manage daily or academic tasks efficiently.
- This includes:
    - Adding new tasks
    - Updating existing tasks
    - Deleting completed or unnecessary tasks

**4. View Notifications**

- The user receives real-time updates and alerts.
- Notifications may include task reminders or important announcements.

**5. Access Campus Services**

- The user can access various campus-related features and utilities from a single platform.

# CHAPTER 4

# METHODOLOGY

## 4.1 Working Principle

The Campus Life OS system works on a client-server model using a microservices architecture. The user interacts with the system through a web-based frontend interface. When a user performs any action (such as login or task creation), the request is sent to the API Gateway.

The API Gateway acts as an intermediary and forwards the request to the appropriate backend microservice. Each microservice processes the request independently and interacts with the database if required. Once the operation is completed, the response is sent back through the API Gateway to the frontend, where it is displayed to the user.

This approach ensures efficient processing, scalability, and smooth communication between different components of the system.

## 4.2 Process Flow

The overall process flow of the system is as follows:

1. The user accesses the application through the frontend interface.
2. The user performs an action (e.g., login, add task, view notifications).
3. The frontend sends an API request to the API Gateway.
4. The API Gateway identifies the appropriate microservice.
5. The request is forwarded to the respective backend service.
6. The microservice processes the request and interacts with the database.
7. The result is sent back to the API Gateway.
8. The API Gateway returns the response to the frontend.
9. The frontend displays the result to the user.

This structured flow ensures smooth and efficient system operation.

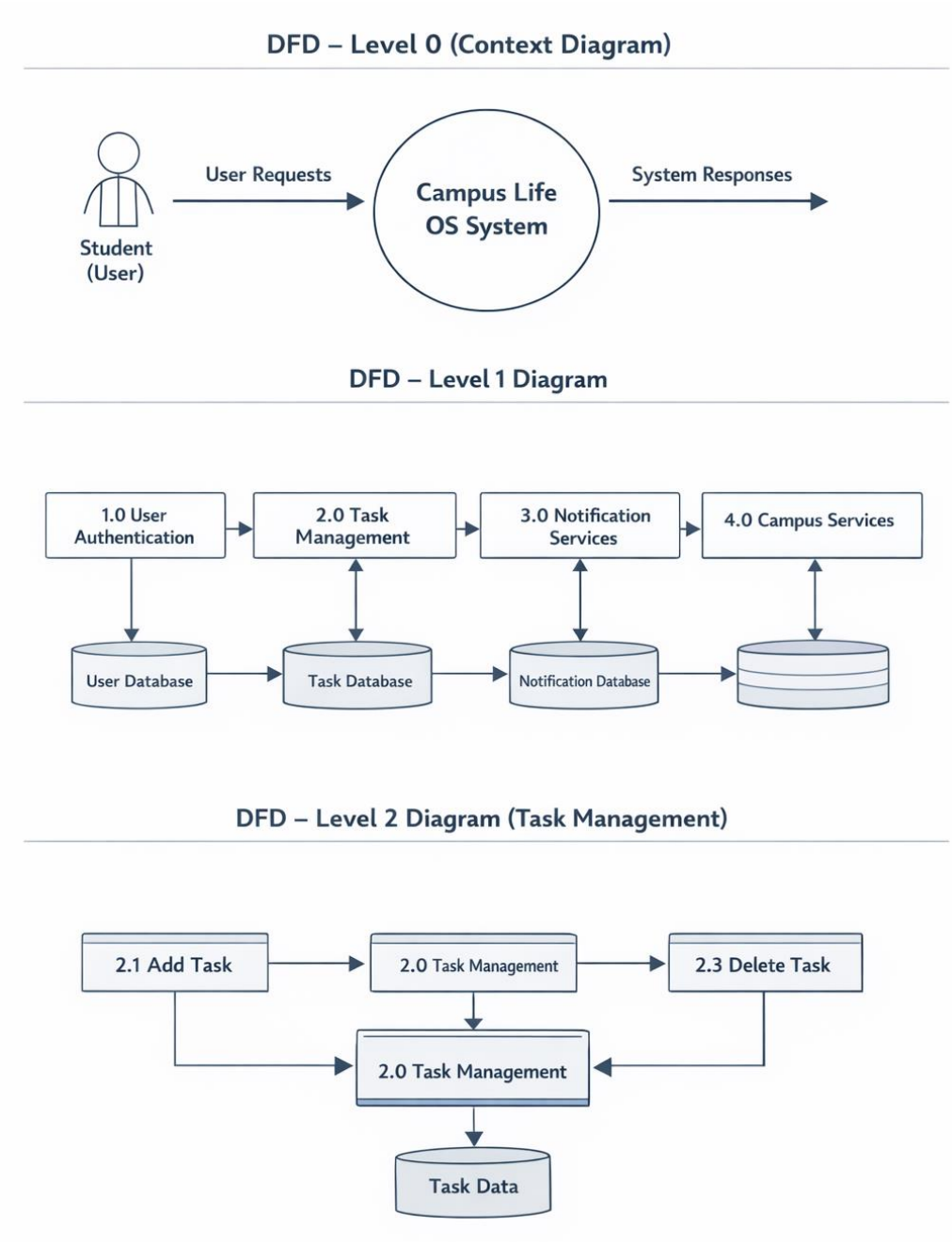



**Figure 4.2.1 Data Flow Diagram**

## 4.3 Module Description

The *Campus Life OS* system is divided into several modules, each responsible for a specific functionality. This modular approach improves system organization, scalability, and maintainability.

**1. Authentication Module**

- Handles user registration and login
- Verifies user credentials securely
- Manages user sessions and access control

**2. Task Management Module**

- Allows users to create, update, and delete tasks
- Helps students organize academic and personal activities
- Tracks task status (completed/pending)

**3. Notification Module**

- Sends real-time alerts and reminders
- Notifies users about deadlines, updates, and important events
- Enhances user awareness and time management

**4. API Gateway Module**

- Acts as a central communication point between frontend and backend
- Routes requests to appropriate microservices
- Handles authentication and request management

**5. Database Module**

- Stores user information, tasks, and notifications
- Ensures data consistency and security
- Supports efficient data retrieval and storage

**6. Docker Module**

- Containerizes each service
- Ensures consistent deployment across environments
- Simplifies scaling and maintenance

## 4.4 Error Handling and Validation

- Error handling and validation are crucial for ensuring system reliability, security, and user satisfaction.
- **1. Input Validation**
  - Validates user inputs at both frontend and backend
  - Ensures correct formats (e.g., email, password)
  - Prevents empty or invalid data submission
- **2. Error Handling**
  - Displays meaningful error messages to users

- Handles system failures gracefully
- Prevents application crashes

- **3. Backend Validation**
  - Ensures data integrity before processing requests
  - Verifies authentication and authorization
  - Prevents invalid operations
- **4. API Error Handling**
  - API Gateway manages request and response errors
  - Returns proper HTTP status codes (e.g., 404, 500)
  - Ensures smooth communication between services
- **5. Security Measures**
  - Prevents unauthorized access
  - Protects sensitive user data
  - Uses authentication and authorization mechanisms

# CHAPTER 5

# EXECUTION

## 5.1 Execution Steps

The execution of the *Campus Life OS* project involves setting up the development environment, running the frontend and backend services, and ensuring proper communication between all components.

**Step 1: Install Required Software**

- Install Node.js
- Install Docker (if using containerization)
- Install a code editor (e.g., Visual Studio Code)

**Step 2: Clone the Project**

- Download or clone the project from the repository
- Open the project folder in the code editor

**Step 3: Install Dependencies**

- Navigate to each service (frontend and backend)
- Install required packages using package manager

**Step 4: Configure Environment Variables**

- Set up .env files for each service
- Add necessary configurations such as database URL, ports, and API keys

**Step 5: Start Backend Services**

- Run each microservice individually
- Ensure all services are running on their respective ports

**Step 6: Start API Gateway**

- Run the API Gateway to handle all incoming requests

**Step 7: Start Frontend**

- Run the frontend application
- Access the system through a web browser (e.g., http://localhost:3000)

**Step 8: Test the Application**

- Register a new user
- Log in and test features like task management and notifications

## 5.2 Compilation & Execution Commands Backend:

**1. Install Dependencies**

npm install

**2. Run Backend Services**

(Execute inside each service folder)

npm start

or

npm run dev

**3. Run API Gateway**

cd api-gateway
npm install
npm start

**4. Run Frontend**

cd frontend
npm install
npm run dev

**5. Using Docker (Optional)**

**Build and Run Containers**

docker-compose up --build

**Stop Containers**

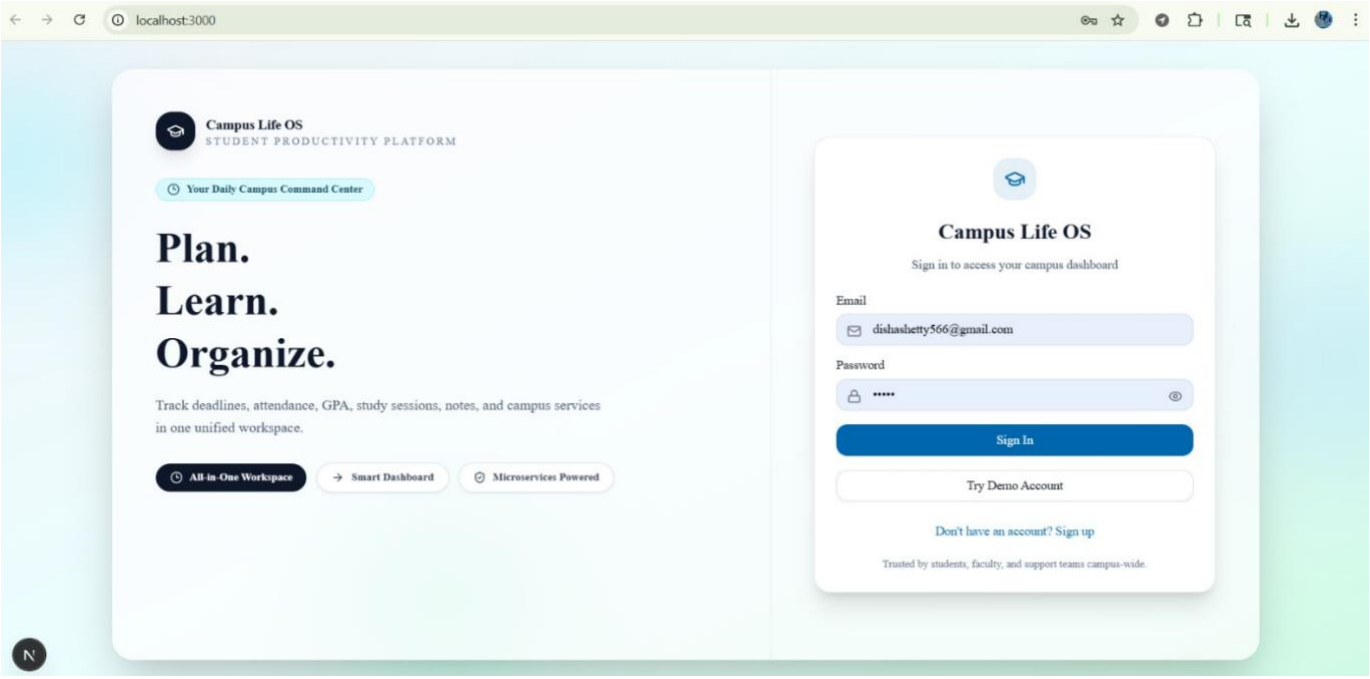
docker-compose down

**Output**

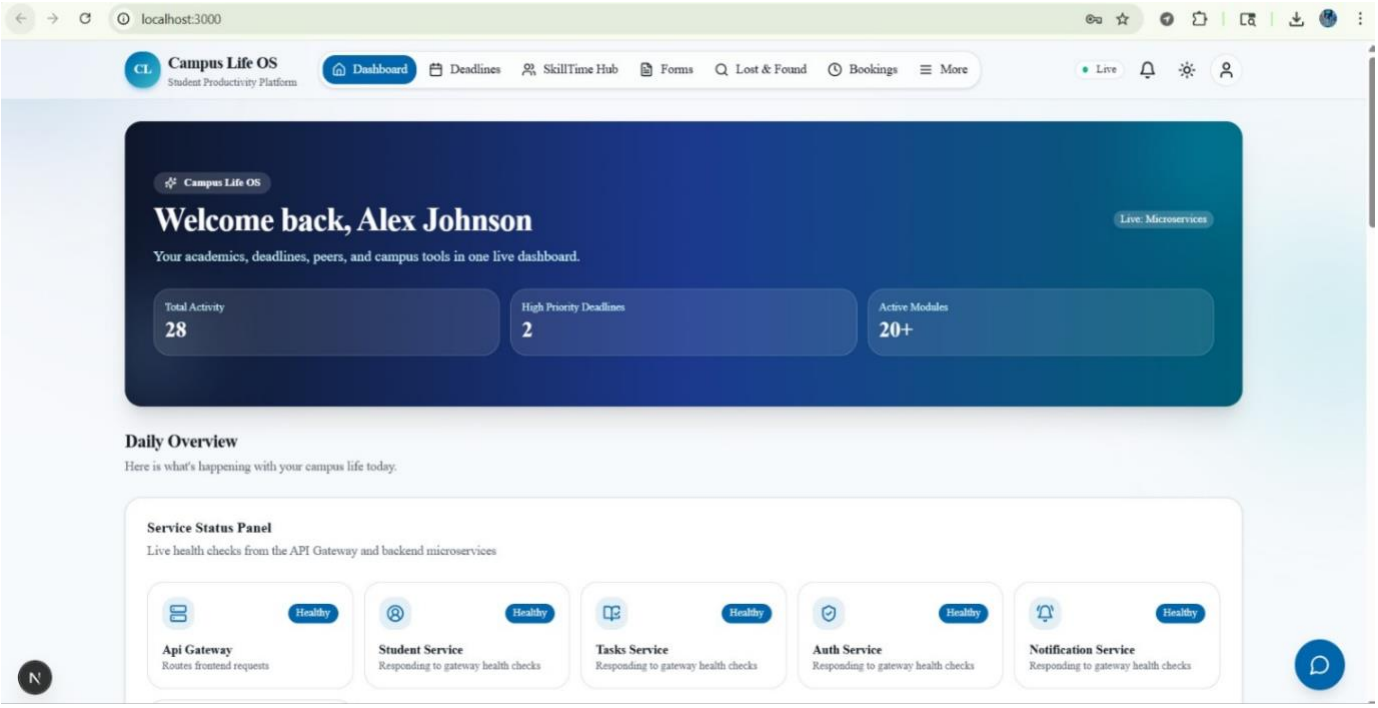
- The application runs on a local server
- Users can access the system via browser
- All modules (login, tasks, notifications) function properly

# CHAPTER 6

# RESULTS




6.1 Login page




6.2 Dashboard Page

# CHAPTER 7

# CONCLUSION AND FUTURE SCOPE

## 7.1 Conclusion

The *Campus Life OS* project successfully demonstrates the development of a modern full-stack web application that integrates multiple campus-related services into a single platformThe project is implemented using advanced technologies such as Next.js, React, TypeScript, Node.js, and Express, along with a microservices architecture. This architecture improves scalability, flexibility, and maintainability by dividing the system into independent services. The use of an API Gateway ensures smooth communication between the frontend and backend, while Docker enables efficient deployment and consistency across environments.The system provides essential functionalities such as user authentication, task management, and notifications, which help students organize their academic and daily activities effectively. The developed application is reliable, efficient, and follows real-world software development practices.

Overall, the project meets its objectives and serves as a strong demonstration of full-stack development and microservices-based system design.

## 7.2 Future Enhancements

Although the system is fully functional, several enhancements can be made in the future to improve its features and usability:

- **Mobile Application Development:** Create Android/iOS apps for better accessibility
- **AI-Based Recommendations:** Provide smart suggestions for task scheduling and productivity
- **Real-Time Chat System:** Enable communication between students and faculty
- **Advanced Analytics Dashboard:** Show performance insights, attendance tracking, and GPA analysis
- **Integration with College Systems:** Connect with official portals for seamless data access
- **Cloud Deployment:** Host the system on cloud platforms for better scalability and availability
- **Enhanced Security Features:** Implement advanced authentication like OTP and biometric login
- **Offline Support:** Allow limited functionality without internet access

# CHAPTER 8

## REFERENCES


1. Node.js Official Documentation
   https://nodejs.org

2. Express.js Official Documentation
   https://expressjs.com

3. Next.js Official Documentation
   https://nextjs.org

4. React.js Official Documentation
   https://react.dev

5. TypeScript Official Documentation
   https://www.typescriptlang.org

6. MongoDB Official Documentation
   https://www.mongodb.com

7. Docker Official Documentation
   https://docs.docker.com

8. MongoDB + Express + Node.js REST API Tutorial
   https://www.mongodb.com/features/express-mongodb-rest-api-tutorial